

From JVM to GPUs

Building GPU-Accelerated AI Libraries
in Java with TornadoVM

Thanos Stratikopoulos

Research Fellow

Beehive Lab • University of Manchester



4th June 2026

About me



Thanos Stratikopoulos

Research Fellow
The University of Manchester - TornadoVM Team

 athanasios.stratikopoulos@manchester.ac.uk

 <https://www.linkedin.com/in/stratika/>

 @thanos_str

 @stratika.bsky.social/

 github.com/stratika

What is Missing from Java in the AI Era?



Every other language ecosystem has a GPU programming stack - Java doesn't (yet)

	Intel oneAPI	OpenCL	NVIDIA CUDA	AMD ROCm	Apple Metal	Java / JVM
Language	DPC++	C / C++	CUDA C, C++, FORTRAN	HIP C++	Metal Shading Language (MSL)	✗
SDK	Intel oneAPI	OpenCL SDK	NVIDIA CUDA	AMD ROCm	macOS SDK	✗
Compiler	LLVM (dpc++)	LLVM	LLVM (nvcc)	LLVM (hipcc)	Metal Compiler	✗
Generated Code	SPIR-V / VULKAN	OpenCL C / SPIR-V	CUDA PTX / OpenCL C	HIP	Metal GPU Code	✗

The Gap

- ✗ No standard GPU programming model in the JDK
- ✗ No SDK, no compiler, no code generation for accelerators
- ✗ JNI/FFI bridges are fragile, unsafe, and non-portable
- ✗ Parallel Streams and Vector API are CPU-only

Fill this the Gap

- ✓ Pure Java GPU programming model
- ✓ JIT compiler: Java → OpenCL / PTX / SPIR-V / Metal
- ✓ Automatic data transfers & memory management
- ✓ Works with any JDK 21+ distribution

Java Developers in the LLM Era

How Java applications connect to LLMs today



Remote Large Models

Java Application

Quarkus / Spring



LangChain4j

Orchestration



Cloud LLM APIs

OpenAI · Anthropic · Gemini · Mistral

Java Developers in the LLM Era

How Java applications connect to LLMs today

Remote Large Models

Java Application

Quarkus / Spring



LangChain4j

Orchestration



Cloud LLM APIs

OpenAI · Anthropic · Gemini · Mistral

Why run models locally?



Privacy

Data stays local



Security

No external calls



Custom Models

Fine-tuned weights



Offline

No internet needed



Domain-Specific

Specialized tasks

Java Developers in the LLM Era

How Java applications connect to LLMs today

Remote Large Models

Java Application

Quarkus / Spring



LangChain4j

Orchestration



Cloud LLM APIs

OpenAI · Anthropic · Gemini · Mistral

Local Models (On Premise)



CPU Inference

Pure Java

llama3.java

Jlama

gemma4.java

qwen35.java

JVM Wrappers

Ollama (via REST)

ONNX Runtime

llama.cpp (JNI)

vLLM

Why run models locally?



Privacy

Data stays local



Security

No external calls



Custom Models

Fine-tuned weights



Offline

No internet needed



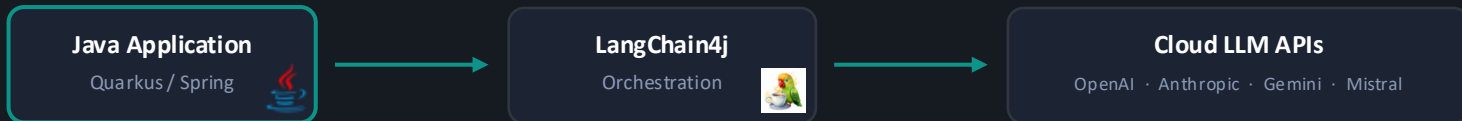
Domain-Specific

Specialized tasks

Java Developers in the LLM Era

How Java applications connect to LLMs today

Remote Large Models



LangChain4j spans both

Local Models (On Premise)



CPU Inference

Pure Java

llama3.java
Jlama
gemma4.java
qwen35.java

JVM Wrappers

Ollama (via REST)
ONNX Runtime
llama.cpp (JNI)
vLLM



GPU Inference

Pure Java

GPULLama3.java

FP16 / Q8 / Q4 quantization
OpenCL · PTX · SPIR-V · Metal

← This talk



Powered by

TORNADO VM

Why run models locally?



Privacy

Data stays local



Security

No external calls



Custom Models

Fine-tuned weights



Offline

No internet needed



Domain-Specific

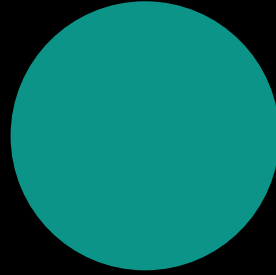
Specialized tasks



Agentic Demo: GPULLama3.java + Quarkus-Langchain4j

*Prompt: "Write a Java class
to print HelloWorld"*





From CPU to GPU Inference in pure modern Java

Understand LLM inference

Transformer Decoder Architecture

Each step's mathematical operation



- 1 Inputs / Embeddings $X = S \cdot W_{\text{embeddings}}$
- 2 Pre RMS LayerNorm $Y = X / \sqrt{(\sum X^2/d + \epsilon)} \cdot w_a$
- 3 Q, K, V Projections $Q, K, V = Y \cdot W_q, Y \cdot W_k, Y \cdot W_v$
- 4 RoPE Embeddings $Q = Q \cdot \cos + Q \cdot R \cdot \sin$
- 5 Multihead Attention $Z = \sigma(Q \cdot K^T / \sqrt{h} + M) \cdot V$
- 6 Residual Connection $X = X + Z \cdot W_o$
- 7 Post RMS LayerNorm $Y = X / \sqrt{(\sum X^2/d + \epsilon)} \cdot w_b$
- 8 MLP / SwiGLU $Z = f(Y \cdot W_{\text{gate}}) \cdot (Y \cdot W_{\text{up}})$
- 9 Residual Connection $X = X + Z \cdot W_{\text{down}}$
- 10 Final RMS LayerNorm $X = X / \sqrt{(\sum X^2/d + \epsilon)} \cdot w_L$
- 11 Logits $X = X \cdot W_{\text{lm_head}}$

Core Operations

RMS LayerNorm

Normalize activations without mean — cheaper than BatchNorm

RoPE

Rotary Position Embedding — encodes position via rotation matrix

Multihead Attention

Scaled dot-product attention with causal mask M

SwiGLU / MLP

Gated FFN: SiLU activation $\times$ gate projection — the largest MatMul

Logits

Final projection to vocabulary size — one MatMul per token

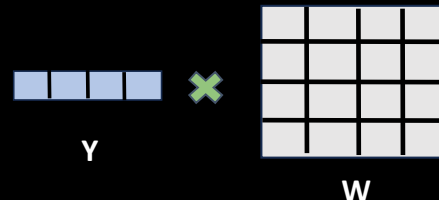
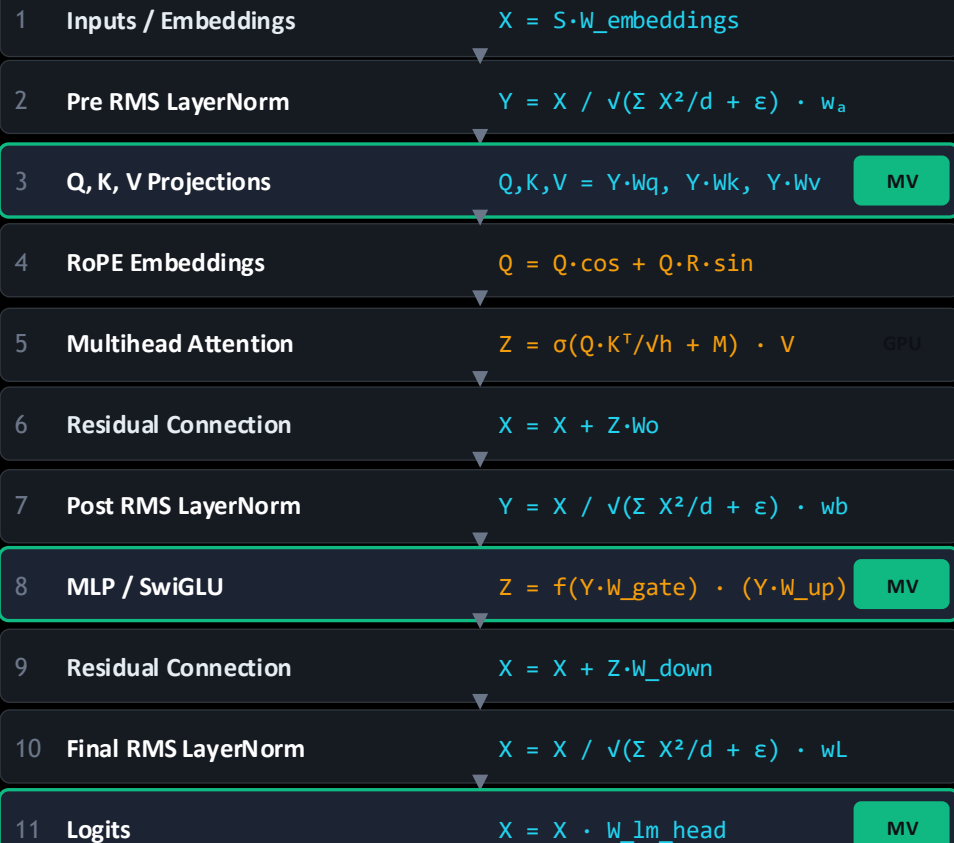
← Repeat L times

Transformer Decoder Architecture

GPU-accelerated operations highlighted — MatMul dominates

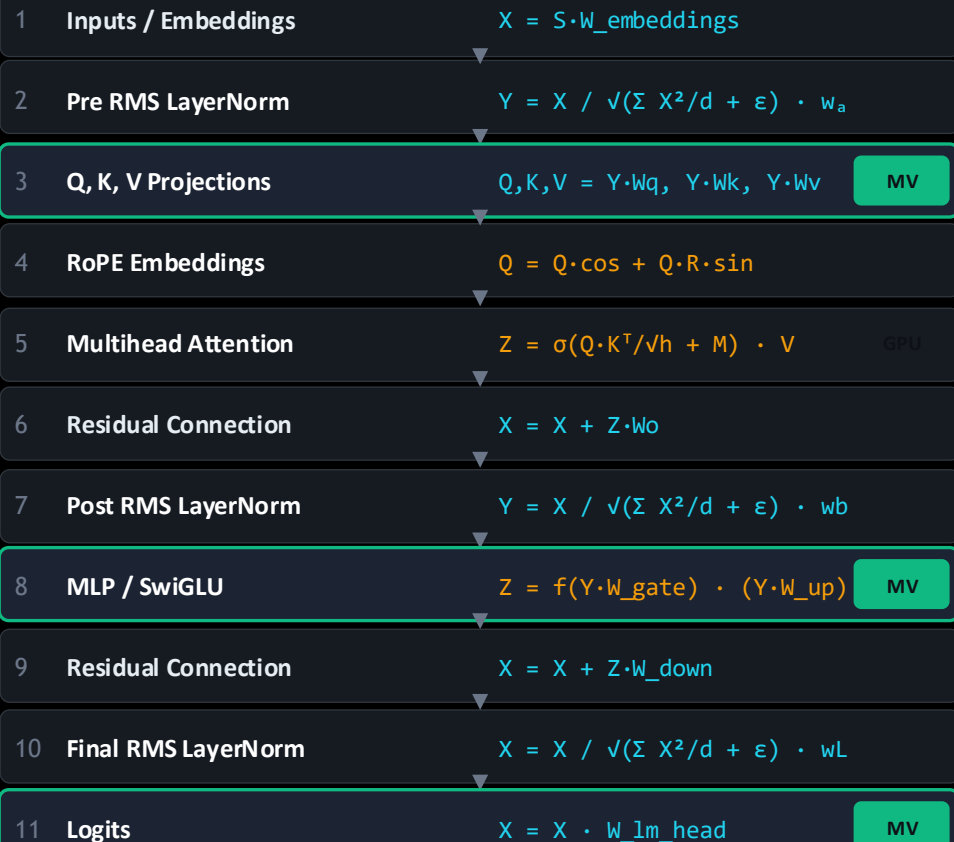


← Repeat L times



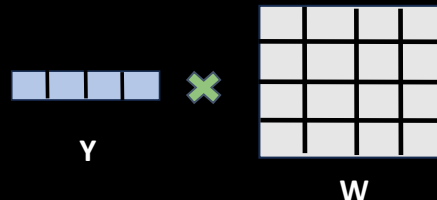
Transformer Decoder Architecture

GPU-accelerated operations highlighted — MatMul dominates



Matrix Vector Multiplication (Compute)

```
public static float[] matrixVectorMultiply(
    float[] weight, float[] y, int rows, int cols) {
    float[] result = new float[rows];
    for (int i = 0; i < rows; i++) {
        result[i] = 0;
        for (int j = 0; j < cols; j++) {
            result[i] += weight[i * cols + j] * y[j];
        }
    }
    return result;
}
```



Transformer Decoder Architecture



GPU-accelerated operations

~81% of all is spent on
Matrix Vector Multiplication

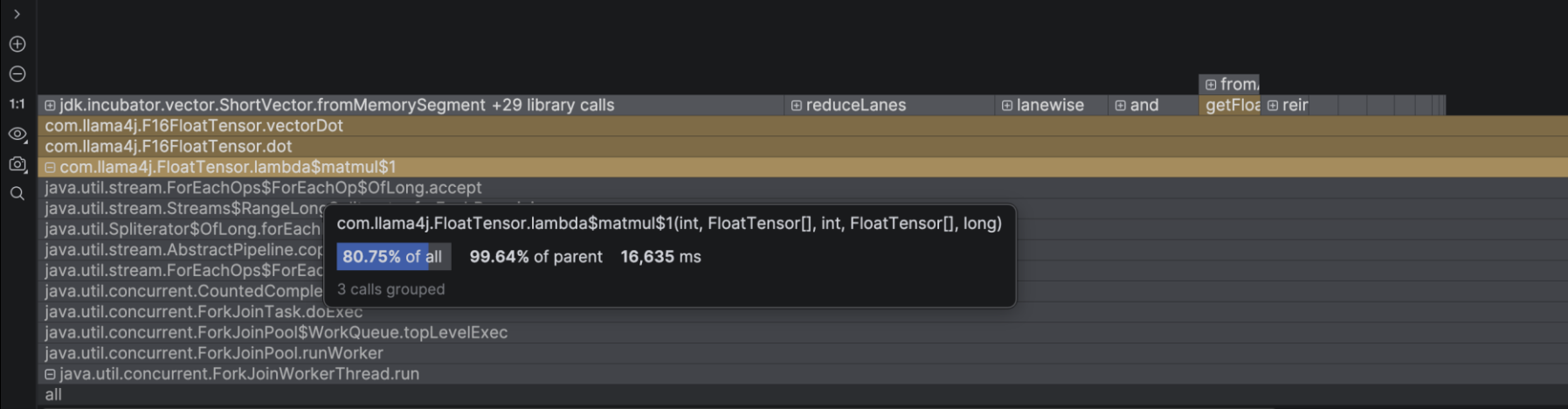
1 Inputs / Embeddings

2 Pre RMS LayerNorm

3 O. K. V Projections

Flame Graph Call Tree Method List Timeline Events

```
vectorMultiply(  
    , int rows, int cols) {  
    rows];  
    ++);  
    Show: CPU Time
```



10 Final RMS LayerNorm

$$X = X / \sqrt{\sum X^2/d + \epsilon} \cdot w_L$$

Y



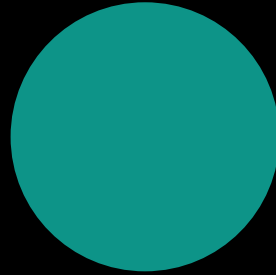
W

11 Logits

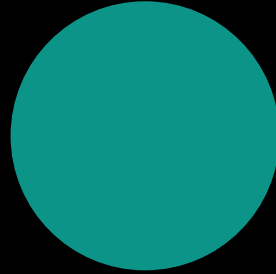
$$X = X \cdot w_{lm_head}$$

MV

← Repeat L times



**How to make these to
run faster?**



TornadoVM

A production-ready, open-source GPU acceleration framework for Java

TornadoVM Ecosystem



JDK 21/25 Distributions

OpenJDK
SapMachine
Azul Zulu
Eclipse Temurin
Red Hat Mandrel
Microsoft JDK
GraalVM

Plugin to JDK

Truffle Framework

GraalVM



JS



TORNADOVM

Hardware Backends



OpenCL



CUDA/PTX



SPIR-V



Metal

Developer Tools



TornadoVMPulse



TornadoViz



NVIDIA NSight Systems

Docker Images



intel

NVIDIA + Intel images

Distribution



Maven Central • SDKMAN!

Integrations



Spring Boot

Quarkus

LangChain4j



Active since 2018, 30+ contributors, 30+ research papers, used by European Space Agency (ESA)

TornadoVM Across the Devovx Stages

From Java bytecode to GPU kernels — Deep dive on internals, API, programming model



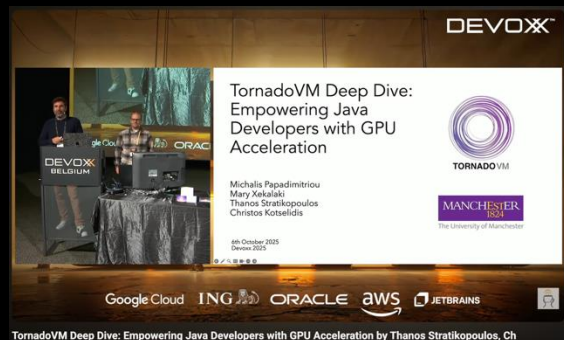
Voxxed Days CERN
(2026)



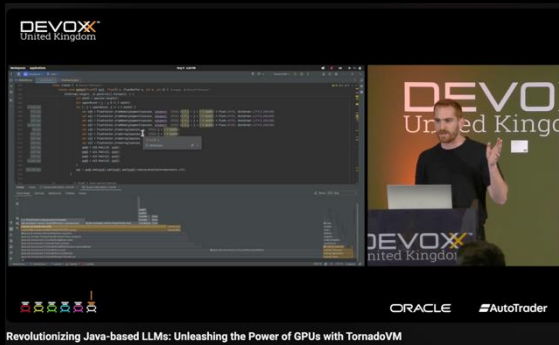
Voxxed Days Ticino
(2026)



Devovx Belgium
(2025)



Devovx Belgium
(2025)



Devovx United Kingdom
(2024)



Devovx Belgium
(2023)

Matrix-Vector Multiply: The Baseline

Step 1 — Plain Java, sequential execution. Where everyone starts.

Sequential Java

```
public static float[] matrixVectorMultiply(
    float[] weight, float[] y, int rows, int cols) {

    float[] result = new float[rows];
    for (int i = 0; i < rows; i++) {
        result[i] = 0;
        for (int j = 0; j < cols; j++) {
            result[i] += weight[i * cols + j] * y[j];
        }
    }
    return result;
}
```

Characteristics

Single-threaded

One core at a time

Simple & correct

Easy to reason about

Slow on large data

$O(\text{rows} \times \text{cols})$ sequential

No GPU use

Hardware underutilized

Next step: **parallelize with one annotation** → @Parallel maps the outer loop to GPU threads automatically

Matrix-Vector Multiply: Loop Parallel API

Step 2 — One annotation. TornadoVM maps iterations to GPU threads.

Before: sequential

```
for (int i = 0; i < rows; i++) {  
    float sum = 0.0f;  
    int rowOffset = i * rows;  
    for (int j = 0; j < columns; j++) {  
        sum += weight.get(rowOffset + j)  
            * y.get(j);  
    }  
    hb.set(i, sum);  
}
```



After: **@Parallel** annotation

```
for (@Parallel int i = 0; i < rows; i++) {  
    float sum = 0.0f;  
    int rowOffset = i * rows;  
    for (int j = 0; j < columns; j++) {  
        sum += weight.get(rowOffset + j)  
            * y.get(j);  
    }  
    hb.set(i, sum);  
}
```

One annotation

@Parallel on the induction variable

Compiler does the work

Iterations → GPU threads

Still correct Java

Runs anywhere, even without TornadoVM

Matrix-Vector Multiply: Kernel API

Step 3 — Full GPU control: thread IDs, local memory, barriers, workgroup reduction.

matrixVectorRowMajor — **KernelContext** + local memory + reduction

```
public static float matrixVectorRowMajor(  
    KernelContext context, int localSize,  
    FloatArray x, FloatArray w, int n) {  
    int rowId = context.groupIdx;  
    int localId = context.localIdx;  
    // Allocate local memory for reduction  
    float[] localSum = context.allocateFloatLocalArray(localSize);  
    int rowOffset = rowId * n;  
    // Each thread calculates partial dot product  
    float partialSum = 0.0f;  
    for (int j = localId; j < n; j += localSize) {  
        partialSum += w.get(rowOffset+j) * x.get(j);  
    }  
    localSum[localId] = partialSum;  
    context.localBarrier();  
    // Parallel reduction within workgroup  
    for (int s = localSize/2; s > 0; s >>= 1) {  
        if (localId < s)  
            localSum[localId] += localSum[localId+s];  
        context.localBarrier();  
    }  
    return localSum[0];  
}
```

TornadoVM GPU Primitives Used

context.groupIdx

Workgroup ID — one per matrix row

context.localIdx

Thread ID within the workgroup

allocateFloatLocalArray()

Fast shared memory for reduction

context.localBarrier()

Sync all threads in workgroup

Parallel reduction

Tree-style sum: $O(\log n)$ steps

More control = more performance — especially for memory-bound operations like MatVec in LLM inference

Quick Install TornadoVM

Three ways to get started



1 Through SDKMAN!

Recommended



```
$ sdk list tornadovm
$ sdk install tornadovm 4.0.1-jdk25-openc1
```

2 Download the SDK directly

<https://www.tornadovm.org/downloads>

3 Clone & build from source

```
$ git clone https://github.com/beehive-lab/TornadoVM.git
$ cd TornadoVM
$ ./bin/tornadovm-installer
$ source setvars.sh
```

Available Versions

4.0.1-jdk25-metal
4.0.1-jdk25-openc1
4.0.1-jdk21-metal
4.0.1-jdk21-openc1

latest

Download TornadoVM SDK

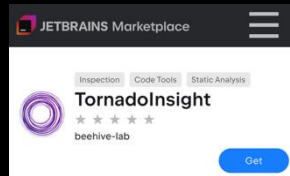
4.0.1 (JDK 21/25) { linux/x86_64
macOS/aarch64
Windows/x86_64

Download

```
Your module-info.java
module my.app {
    requires tornado.api;
}
```



Demo: Quick Install & Run TornadoVM Program & TornadoInsight



GPULlama3.java - tornadovm

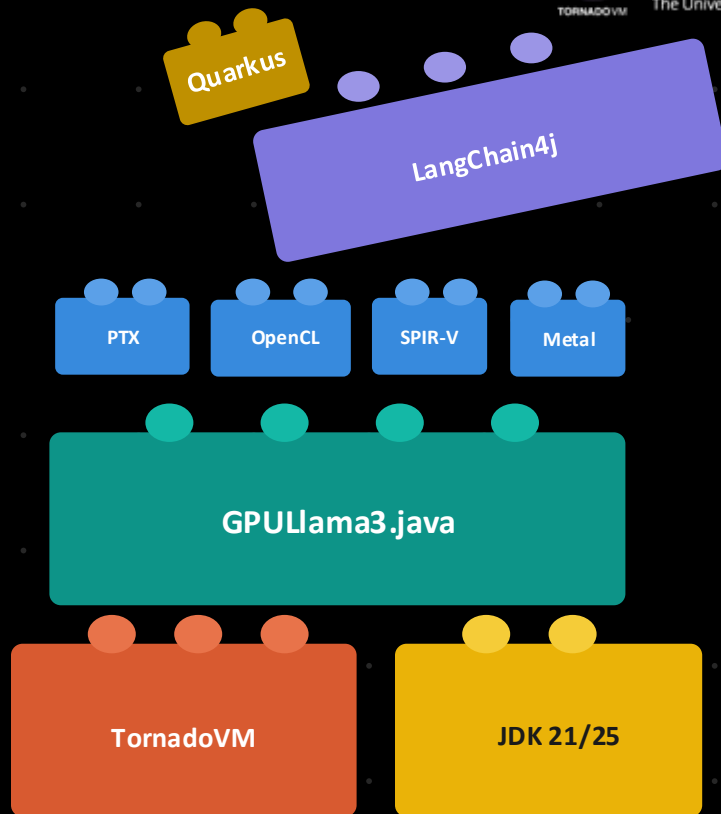
<https://github.com/beehive-lab/GPULlama3.java>



Building GPULLama3.java with TornadoVM

for the AI era

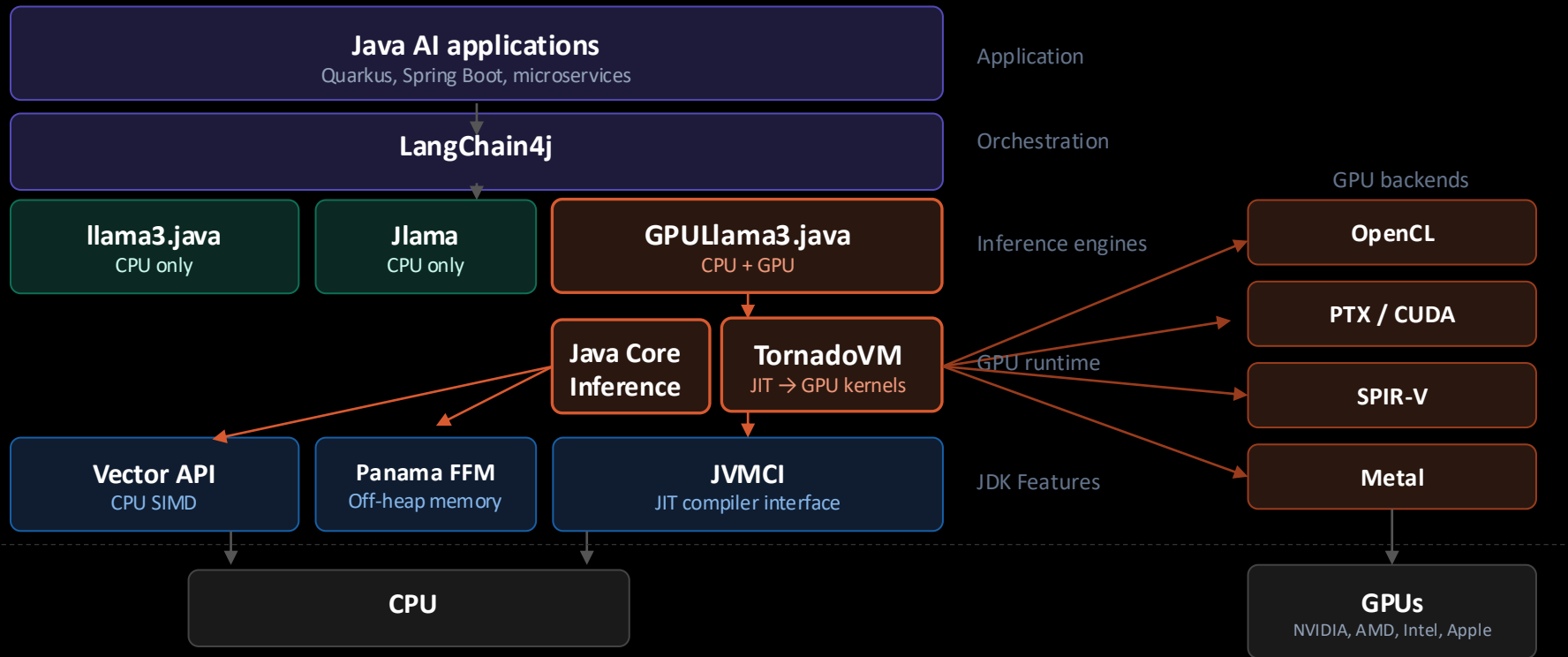
github.com/bee-hive-lab/GPULLama3.java



How it fits in Java Local Inference



CPU engines use Vector API directly — GPULLama3.java routes through TornadoVM to GPU backends



Pure Java GPU inference — no JNI, no Python, no native dependencies

Supported Models & Backends

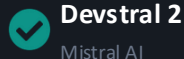
GPULLama3.java — GGUF format, multiple quantization levels, multi-backend

Model Families



Llama 3

Meta



Devstral 2

Mistral AI



Phi-3

Microsoft



IBM Granite 4.0

IBM Research



Mistral

Mistral AI



Qwen 2.5 / 3

Alibaba



IBM Granite 3.2+

IBM Research

More coming...

Community PRs welcome

Quantization

FP16

Half precision · 2 bytes/param

Q8_0

8-bit quantized · 1 byte/param

Q4_0

4-bit quantized · 0.5 bytes/param

GPU Backends

OpenCL

NVIDIA · AMD · Intel · ARM

PTX / CUDA

NVIDIA GPUs — native perf

Metal

Apple Silicon (M1–M5)

Integrated as GPU inference engine in **Quarkus** and **LangChain4j** — full Java AI stack, no Python needed

Community Contributions

Java community getting interested on extend, support and simplify Java GPU LLM inference



Devstral 2 support (Mistral 3 architecture, Tekken tokenizer, YaRN ..

Merged mikepapadim merged 1 commit into `beehive-lab:main` from `AdamBien:main` 2 days ago

Conversation 7 Commits 1 Checks 7 Files changed 19



AdamBien commented 3 days ago

Contributor ...

Devstral 2 support (Mistral 3 architecture, Tekken tokenizer, YaRN RoPE)

Devstral 2 uses the mistral3 architecture with independent head dimensions (head_dim=128 != dim(num_heads=160), requiring non-square Q/K/V projections, a dedicated GPU kernel for precomputed YaRN RoPE frequencies, and a GPT-2 BPE tokenizer (Tekken) instead of Mistral's SentencePiece.

Key changes

- DevstralConfiguration with explicit headDim, qDim() (4096), kvDim() (1024)
- DevstralTokenizer: GPT-2 BPE with BYTE_ENCODER and Tekken pre-tokenization regex
- YaRN RoPE: precomputeFreqsCiYaRN with mscale and ramp interpolation
- GPU kernels: fusedQKVMatmulQ8NonSquare, ropeRotationWithCacheCopyPrecomputed
- Devstral-specific FFN layers, layer planners, state, chat format
- GGUF metadata: mistral3* prefix, attention.key_length for head_dim

Add Q4_K/Q5_K/Q6_K GPU support via Q8_0 dequantization #108

Open AdamBien wants to merge 2 commits into `beehive-lab:main` from `AdamBien:main`

Conversation 1 Commits 2 Checks 6 Files changed 13



AdamBien commented yesterday

Contributor ...

- Add GPU support for K-quant models (Q4_K_M, Q5_K_M, Q6_K) via load-time dequantization to Q8_0
- New FloatTensor implementations: Q4_KFloatTensor, Q5_KFloatTensor, Q6_KFloatTensor
- Dequantization correctly handles TornadoVM's 16-byte ARRAY_HEADER memory layout
- Centralize weight loading log message in AbstractModelLoader (shows actual model quantization, e.g. "Q4_K_M -> Q8_0")

Add Apple Metal backend support #103

Open ArturSkowronski wants to merge 1 commit into `beehive-lab:main` from `ArturSkowronski:feat/metal-backend-support`

Conversation 4 Commits 1 Checks 6 Files changed 1



ArturSkowronski commented last week · edited ·

First-time contributor ...

Add Apple Metal backend support to the `llama-tornado` launcher, enabling GPUllama3 to run on macOS with TornadoVM's native Metal driver (shipped in TornadoVM 4.0+).

Changes (launcher only, no Java code changes):

- Add `METAL` variant to the `Backend` enum
- Add `--metal` CLI flag for backend selection
- Configure Metal-specific module path (`tornado.drivers.metal`) and export list (`metal-exports`)

The TornadoVM API is backend-agnostic, so the Java inference code works without modification - only the launcher needed updating.

Motivation

TornadoVM 4.0 shipped a Metal backend ([PR #736](#)), but `llama-tornado` only supported `--opencl` and `--gtx`. The GPUllama3 README already notes:

"TornadoVM does not have a Metal backend yet [...] until we add a Metal backend to TornadoVM and start optimizing it."

TornadoVM 4.0 has now added it - this PR enables GPUllama3 to use it 🎉



Add JBang catalog and local usage examples to README.md #91

Merged mikepapadim merged 1 commit into `main` from `docks/readme-jbang` on Dec 16, 2025

Conversation 0 Commits 1 Checks 6 Files changed 1



Add llamaTornado Java 25 single-file launcher with Metal backend support #105

Merged mikepapadim merged 3 commits into `beehive-lab:main` from `AdamBien:main`

4 days ago

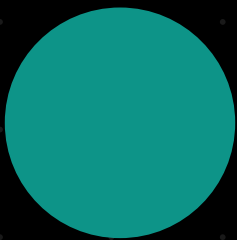


Add CI Actions for Quarkus-LangChain4j integration #89

Merged mikepapadim merged 19 commits into `main` from `ci/quarkus-langchain4j-IT` 2 weeks ago

Conversation 3 Commits 19 Checks 7 Files changed 1





Bringing Advanced GPU Features From LLM Frameworks into Pure Java

Kernel Fusion
19 → 8 kernels

Flash Attention
 $O(N^2) \rightarrow O(N)$

CUDA Graphs
2.11× speedup

Prefill Batching
3.55× over CPU

Tensor Cores
mma.sync from Java

GPU optimization knowledge transfers — the language doesn't have to be C++ anymore

How We Build GPU-enabled LLMs in Pure Java



GPU optimization knowledge transfers — the language doesn't have to be C++ anymore



GPU Optimization Concepts

- Tiled matrix multiplication
- Shared/local memory usage
- Flash Attention (online softmax)
- Kernel fusion strategies
- Prefill/decode separation
- Memory coalescing patterns
- CUDA Graphs for replay



Pure Java via TornadoVM

- `KernelContext` for GPU primitives
- `allocateFloatLocalArray()`
- `localBarrier()/globalBarrier()`
- `HalfFloatArray` (FP16 native)
- `TaskGraph` → fused kernels
- `persistOnDevice()` for KV-cache
- `withCUDAGraph()` for replay



High-performance LLM Inference

- Multiple Models
- Multiple Backends
- Scalable across multiple devices

The Optimization Pipeline

Known GPU optimizations → pure Java implementation via TornadoVM → High-performance LLM Inference in Java

Each new optimization (kernel fusion, Flash Attention, prefill/decode) follows this pattern.

Writing Efficient GPU Code in Java

Each transformer operation $\rightarrow$ a TornadoVM `.task()` in the TaskGraph

Transformer Ops

TaskGraph `.task()` Mapping

Pre RMS LayerNorm

Q, K, V Projections

RoPE Embeddings

Multihead Attention

Residual + Mat Mul

Post RMS LayerNorm

MLP / SwiGLU

Residual

`.task("reductionsOneBlock", Kernels::reductionOneBlockWithLayer, ...)`

`.task("mapContext", Kernels::reductionOneBlock2WithLayer, ...)`

`.task("qmatmul", Kernels::matrixVectorGeneric, ...)`

`.task("kmatmul", Kernels::matrixVectorGeneric, ...)`

`.task("vmatmul", Kernels::matrixVectorGeneric, ...)`

`.task("rope", Kernels::ropeRotation, ...)`

`.task("copyToCaches", Kernels::copyToCache, ...)`

`.task("parallel-attention", Kernels::processHeadsParallel, ...)`

`.task("matmul1", Kernels::matrixVectorGenericWithResidual, ...)`

`.task("reductionsOneBlockFFN", Kernels::reductionOneBlockWithLayer, ...)`

`.task("fused_ffn_w1_w3", Kernels::fusedFeedForwardWithSiLUAndGLUActivation, ...)`

`.task("projectionTwo", Kernels::matrixVectorGenericWithResidual, ...)`

12 tasks per layer • each a GPU kernel • `fused_ffn_w1_w3` = 5 ops in 1 kernel • `TornadoVMLayerPlanner.java`

Advanced GPU Code: The Fused Kernel

One kernel, many operations — with HalfFloatArray (FP16) for memory efficiency

fusedFeedForwardWithSiLUAndGLUActivation

```
public static void fusedFeedForwardWithSiLUAndGLUActivation(
    KernelContext context, FloatArray x, FloatArray hb,
    HalfFloatArray w1, HalfFloatArray w3,
    int n, int d, int localWorkGroupSize) {
    int rowId = context.groupIdx;
    int localId = context.localIdx;

    // Two matrix-vector products in parallel
    float sum1 = matrixVectorRowMajorOptimized(context, localWorkGroupSize, x, w1, n);
    float sum3 = matrixVectorRowMajorOptimized(context, localWorkGroupSize, x, w3, n);

    if (localId == 0) {
        float silu = siluActivation(sum1); // SiLU:  $x * \sigma(x)$ 
        float result = silu * sum3; // GLU gating
        hb.set(rowId, result);
    }
}
```



Linear projection W1



Linear projection W3



SiLU activation



GLU gating



Store output

Without fusion: **5 kernel launches + 4 intermediate buffers** → With fusion: **1 kernel launch, everything in registers**

Automatic memory Management

Critical for LLM inference — weights, KV-cache, activations

FIRST_EXECUTION

Copy once — model weights

EVERY_EXECUTION

Copy each call — input tokens

UNDER_DEMAND

Manual — large outputs

`persistOnDevice()`

Keep on GPU — KV-cache

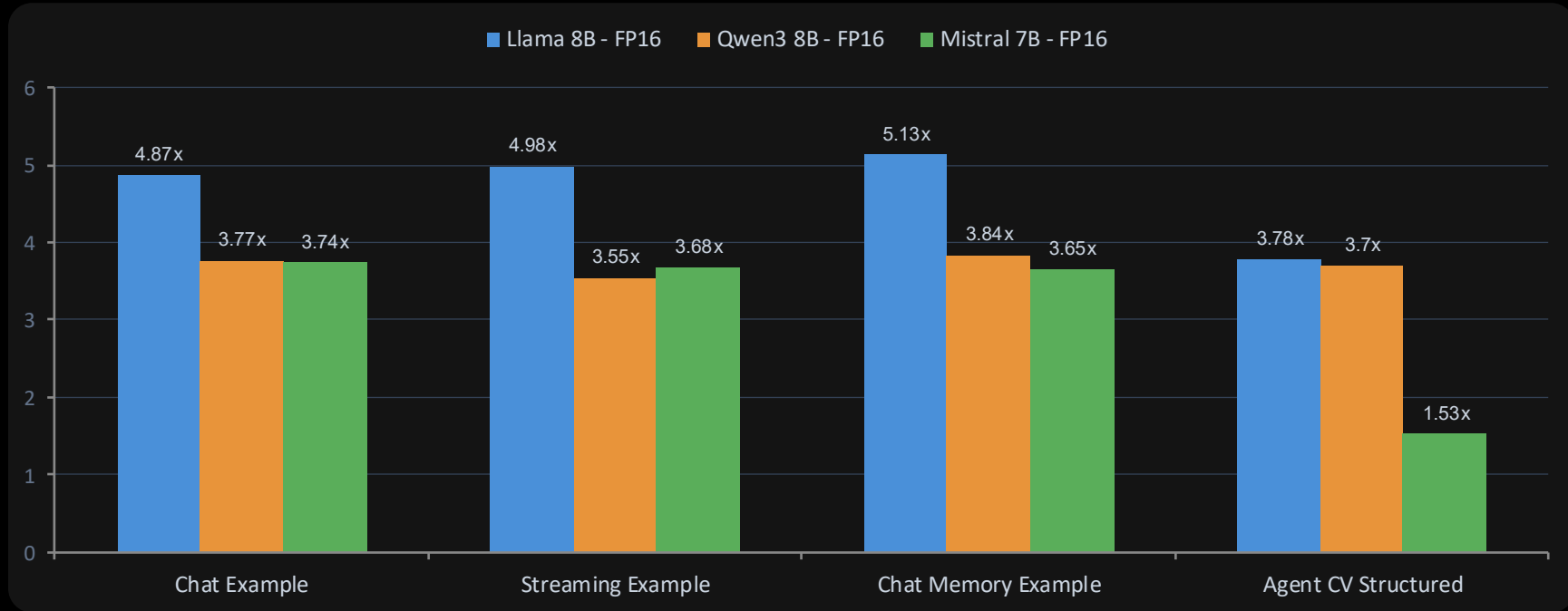
```
// LLM inference memory strategy
TaskGraph tg = new TaskGraph("transformer_layer")
    .transferToDevice(DataTransferMode.FIRST_EXECUTION, weights) // 7B params — copy once
    .transferToDevice(DataTransferMode.EVERY_EXECUTION, input)   // token embeddings
    .task("attn", Attention::forward, ctx, weights, input, kvCache, output, ...)
    .persistOnDevice(kvCache)                                   // keep KV-cache on GPU
    .transferToHost(DataTransferMode.EVERY_EXECUTION, output);  // logits back

// Partial transfers for large outputs
DataRange range = new DataRange(output).withOffset(0).withSize(vocabSize);
result.transferToHost(range); // only copy what's needed
```

GPU Speedup Over pure Java CPU



GPULLama3.java (TornadoVM OpenCL) vs CPU-only inference • Higher is better



GPU: GPULLama3.java (TornadoVM OpenCL) • NVIDIA GeForce RTX 5090 Mobile - 24 GB

CPU: GPULLama3.java (llama3.java, parallel streams, Vector API) • Intel® Core™ Ultra 9 275HX × 24, 64 GB

More Optimizations: Prefill/Decode Batching

Batch prompt tokens during prefill, sequential during decode — up to 3.55× over CPU baseline



Prefill Phase

All prompt tokens known upfront → batch them (e.g., 32)

Skip logits for all but last prompt token

Replaces sequential one-by-one ingestion



Decode Phase

Sequential — each token depends on previous

Same as current behavior, no change needed

Performance — Llama 1B FP16 · RTX 5090 · Batch=32

Short Prompt

CPU standard		22.24 tok/s
CPU batched prefill		27.30 tok/s
GPU standard		61.61 tok/s
GPU prefill/decode		62.79 tok/s
GPU batched prefill		68.01 tok/s

Long Prompt

CPU standard		24.50 tok/s
CPU batched prefill		39.69 tok/s
GPU standard		55.69 tok/s
GPU prefill/decode		59.51 tok/s
GPU batched prefill		87.02 tok/s

Long prompts: **3.55× GPU vs CPU baseline** • Short prompts: **3.06× GPU vs CPU baseline** • [PR #102](#)

Advanced CUDA Features in Java

Three CUDA performance techniques — all from pure Java via TornadoVM API



CUDA Graphs



Up to 2.11x more tokens

Warp SIMD Ininsics

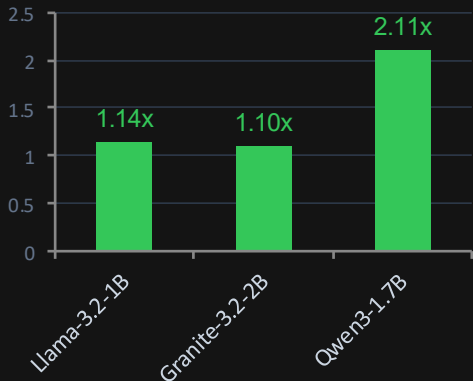


`shfl.sync.down`

Tensor Core MMA



`mma.sync.m16n8k16`



```
// Warp reduction — one call
float sum = ctx.simdSum(value);

// Manual butterfly pattern
value += ctx.simdShuffleDown(value, 16);
value += ctx.simdShuffleDown(value, 8);
value += ctx.simdShuffleDown(value, 4);
value += ctx.simdShuffleDown(value, 2);
value += ctx.simdShuffleDown(value, 1);

// Broadcast lane 0 to all
float shared = ctx.simdBroadcastFirst(value);
```

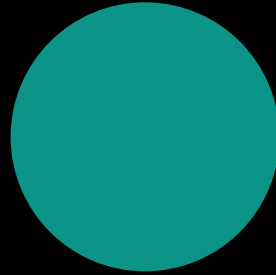
- `simdSum`: 5x `shfl.sync.down` + 5x `add.f32`
- `simdShuffleDown`: `shfl.sync.down.b32`
- `simdBroadcastFirst`: `shfl.sync.idx.b32`

No shared memory, no local barrier, warp-collective

```
void gemm(KernelContext ctx, ...) {
    float[] fragC = ctx.mmaFragment();
    HalfFloat[] fragA = ctx.mmaLoadA(aTile, tileK);
    HalfFloat[] fragB = ctx.mmaLoadB(bTile, tileK);

    MMAShape shape = MMAShape.M16N8K16;
    fragC = ctx.mma(fragA, fragB, fragC, shape);
    ctx.mmaStore(fragC, out, row, col, stride);
}
```

- **mmaFragment**: 4x `mov.f32`:
allocates and initializes registers
- **mmaLoadA/B**: 4x/2x `ld.shared.b32`:
loads fragment into registers
- **mma**: `mma.sync.aligned.m16n8k16`:
one instruction, 32 lanes
- **mmaStore**: 4x `st.global.f32`:
writes fragment to global memory



Integrations

LangChain4j, Quarkus & the Java AI ecosystem

Official Integrations

GPULLama3.java as a first-class model provider in both ecosystems



LangChain4j



The screenshot shows the LangChain4j documentation website. The left sidebar has a navigation menu with 'Introduction', 'Get Started', 'Tutorials', 'Integrations', and 'Language Models'. The main content area is titled 'GPULLama3.java' and includes a description: 'GPULLama3.java builds on TornadoVM to leverage GPU and heterogeneous computing for faster LLM inference directly from Java. Currently, GPULLama3.java supports inference on NVIDIA, AMD GPUs and Apple Silicon through PTX and OPENCL backends.' Below this is a 'Project setup' section with instructions on how to add the dependency to a Maven project.

docs.langchain4j.dev/integrations/language-models/gpullama3-java

Quarkus-LangChain4j



The screenshot shows the Quarkiverse documentation website. The left sidebar has a navigation menu with 'LangChain4j', 'Getting Started', 'Guides', 'Reference', 'AI Services', and 'Models'. The main content area is titled 'GPULLama3.java Chat Models' and includes a description: 'GPULLama3.java provides a Java-native implementation of LLMs that runs entirely in Java and executes automatically on GPUs via TornadoVM.' Below this is a 'Prerequisites' section with the text 'Java Version and TornadoVM SDK'.

docs.quarkiverse.io/quarkus-langchain4j/dev/gpullama3-chat-model.html

Drop-in GPU inference provider — same API, same tooling, transparent GPU acceleration

Works with RAG pipelines, agents, memory, tool calling — full Java AI stack, no Python needed

Using a Quarkus-LangChain4j example

Quarkus-LangChain4j



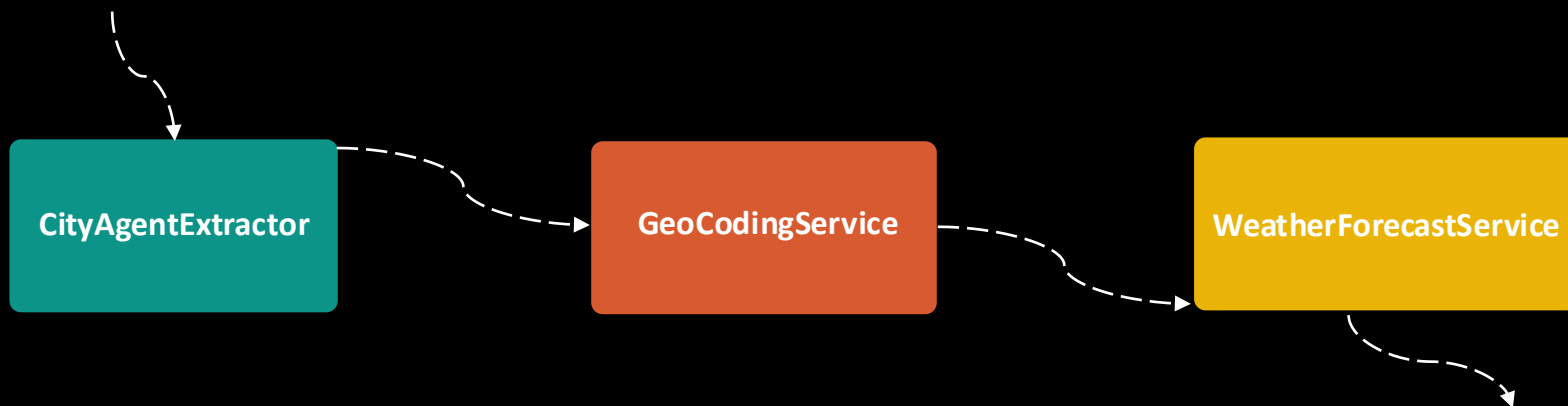
1



Weather Demo - Agentic Demo for Weather Prediction

<https://github.com/quarkiverse/quarkus-langchain4j/tree/main/samples/weather-agent>

http://localhost:8080/weather?city=Sofia



Response:

"The weather in Sofia today is ..."

Using a Quarkus-LangChain4j example



Quarkus-LangChain4j



1



Weather Demo - Agentic Demo for Weather Prediction

<https://github.com/quarkiverse/quarkus-langchain4j/tree/main/samples/weather-agent>

2

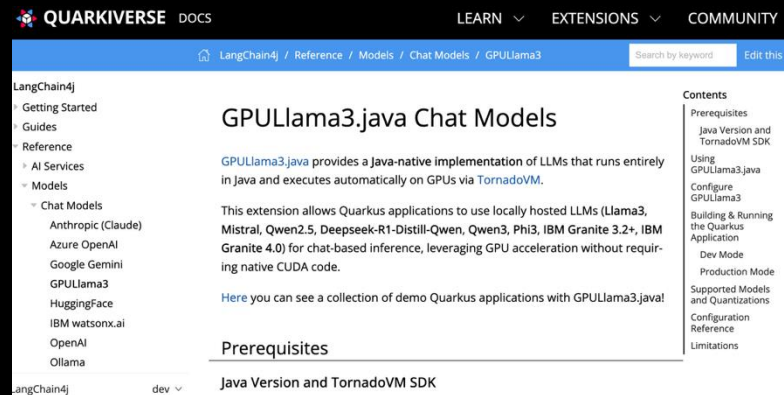
Update dependencies in pom file to use the `quarkus-langchain4j-gpu-llama3` artifact

Using a Quarkus-LangChain4j example

Quarkus-LangChain4j



- 1  Weather Demo - Agentic Demo for Weather Prediction
<https://github.com/quarkiverse/quarkus-langchain4j/tree/main/samples/weather-agent>
- 2 Update dependencies in pom file to use the `quarkus-langchain4j-gpu-llama3` artifact
- 3 Update the model configuration in the `application properties`



docs.quarkiverse.io/quarkus-langchain4j/dev/gpullama3-chat-model.html

Using a Quarkus-LangChain4j example



Quarkus-LangChain4j



1



Weather Demo - Agentic Demo for Weather Prediction

<https://github.com/quarkiverse/quarkus-langchain4j/tree/main/samples/weather-agent>

2

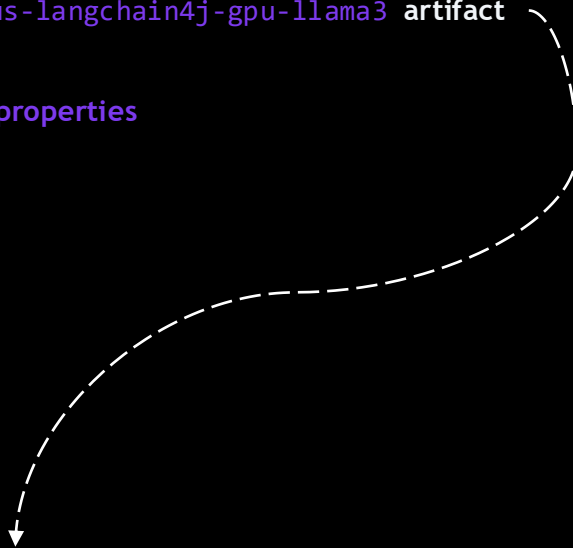
Update dependencies in pom file to use the `quarkus-langchain4j-gpu-llama3` artifact

3

Update the model configuration in the `application properties`

4

Build and run!



Weather Demo - Agentic Demo for Weather Prediction

<https://github.com/stratika/quarkus-langchain4j/tree/gpullama3/weather-agent>



Demo 3: Quarkus + GPULLama3.java inference

*http://localhost:8080/
weather?city=Sofia*

CityAgentExtractor

GeoCodingService

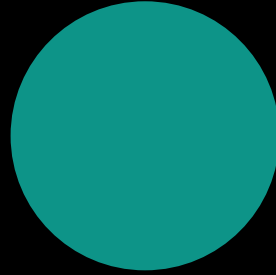
WeatherForecastService

*Response:
"The weather in Sofia
today is ..."*



Weather Demo - Agentic Demo for Weather Prediction

<https://github.com/stratika/quarkus-langchain4j/tree/gpullama3/weather-agent>



Takeaways & Future

What's Next — Becoming a vLLM in Java

Four modern LLM serving goals — reinterpreted through the JVM



Continuous Batching via Virtual Threads

vLLM's continuous batching

Project Loom meets LLM serving — each request is a virtual thread; the scheduler batches tokens across them transparently.

- Structured concurrency for request lifecycle
- No thread-pool tuning — millions of VTs
- Natural fit for Quarkus reactive endpoints



Off-Heap Paged KV- Cache + CUDA Unified Memory

vLLM's PageAttention

Panama MemorySegment for GC-free pages, backed by CUDA Unified Memory — same pointer works on CPU and GPU, zero explicit transfers.

- FFM API: off-heap, zero-copy semantics
- Unified address space: no cudaMemcpy
- Page-faulted on-demand access



Multi-Device Orchestration via TaskGraph

vLLM's tensor parallelism

Shard transformer layers across NVIDIA + Apple Silicon + Intel in one JVM using `withConcurrentDevices()` and multi-TaskGraph composition.

- Cross-vendor — no CUDA lock-in
- One Java app, many accelerators
- TornadoVM handles device selection



Advanced NVIDIA GPU Support

vLLM's CUDA kernels

Target tensor cores via PTX `mma.sync`, fused TF32/FP16/INT8 paths, and cuBLAS / cuDNN interop — all from Java via TornadoVM's PTX backend.

- `mma.sync` instructions from Java kernels
- cuBLAS / cuDNN library interop
- Peak NVIDIA throughput, no JNI glue

The goal: Java-native LLM serving that uses JVM strengths Python can't — TornadoVM, Panama, multi-backend runtime.

Key Takeaways & What's Next



1

Java's GPU inference gap is closed

GPUllama3.java runs 7+ model families with FP16/Q8/Q4 on OpenCL, PTX & Metal — entirely in pure Java, no JNI, no Python.



2

GPU optimization techniques work in Java

Tiled MatMul, Flash Attention, kernel fusion (19→8 kernels), CUDA Graphs (2.1×), prefill/decode batching (3.55×) — all via TornadoVM's KernelContext.



3

TornadoVM offers efficient memory management for LLM workloads

Data can be transferred once or on demand.



4

Production-ready today

Quarkus & LangChain4j integration — enterprise Java teams can deploy GPU-accelerated LLM inference without leaving their stack.



5

Growing ecosystem & what's next

Next: continuous batching (Virtual Threads), paged KV-cache (Panama + CUDA UM), multi-device (TaskGraph), tensor cores (PTX mma.sync). Goal: Java-native LLM serving using JVM strengths Python can't match.

Thank you!



Resources



TornadoVM

github.com/bee-hive-lab/TornadoVM



GPULLama3.java

github.com/bee-hive-lab/GPULLama3.java



Docker Image

hub.docker.com/r/bee-hive-lab/gpullama3.java-nvidia-openjdk-opencl



LangChain4j Demo

github.com/bee-hive-lab/gpullama3-langchain4j-demo



Quarkus Demo

github.com/bee-hive-lab/gpullama3-quarkus-langchain4j-demo



SDKMAN!

sdkman.io/sdks/tornadovm

Scan to get started



gpullama3.dev

TornadoVM



<https://www.linkedin.com/company/tornadovm>



[@tornadovm](https://twitter.com/tornadovm)



@tornadovm.org



<https://github.com/bee-hive-lab/TornadoVM>



tornadovmcommunity.slack.com/

Thank you!



encrypt



oneAPI

UK Research
and Innovation

athanasios.stratikopoulos@manchester.ac.uk

<https://www.linkedin.com/in/stratika/>

@thanos_str



@stratika.bsky.social/

github.com/stratika